Lecture 27 — Interval DP & Digit DP

> Year Curriculum · Prerequisites: Week 14–15 (DP I & II), Week 7 (prefix sums), Week 26 (bitmask DP, for comparison of state design) · Estimated time: 2.5 hours

1. Overview

This lecture covers two DP families whose state spaces look unusual at first but follow very regular templates.

- **Interval DP** works on a sequence where every operation acts on adjacent elements or on the ends, so every subproblem is a **contiguous segment** `a[l..r]`. The state is `dp[l][r]`; the transition usually chooses a split point or removes an end. Typical complexity is $O(n^2)$ or $O(n^3)$.
- **Digit DP** counts integers in a huge range `[A, B]` (up to 10^18, or even numbers with thousands of digits given as strings) that satisfy a property depending on their decimal digits. We build the number digit by digit from the most significant one, carrying a small state plus a **tight** flag that remembers whether we are still bounded by the upper limit.

Both appear constantly, from Div. 2 C/D up to regional finals, and both are much easier once the template is internalized.

2. Interval DP

2.1 When to use it

Signals: a row of objects; operations merge **adjacent** objects, delete a contiguous block, or take from the **ends**; after an operation the remaining structure splits into **independent** contiguous parts. Constraints like $n \le 400$–500 suggest $O(n^3)$; $n \le 5000$ suggests $O(n^2)$.

2.2 Merging: AtCoder DP N *Slimes*

$n \le 400$ slimes with sizes `a[i]` stand in a row. Merging two adjacent slimes of sizes x and y creates one slime of size x + y and costs x + y. Find the minimum total cost to end with one slime.

The obvious greedy (always merge the cheapest adjacent pair) is wrong: on `2 2 1 2` it first merges the middle pair `2+1` (cost 3), then pays 5 and 7, total 15, while merging `2+2` and `1+2` first and then `4+3` costs 4 + 3 + 7 = 14.

Think about the last merge. Whatever we do, the final operation merges a slime made from `a[l..k]` with one made from `a[k+1..r]` for some split `k`. These two slimes were built independently (merges never cross the boundary before the last step), and the last merge costs `a[l] + … + a[r]` regardless of `k`. So

```
dp[l][l] = 0
dp[l][r] = min_{l ≤ k < r} (dp[l][k] + dp[k+1][r]) + sum(l, r)
```

with `sum` from prefix sums. There are O(n²) states and O(n) transitions each: O(n³) = 6.4·10^7 for n = 400.

Order of computation. `dp[l][r]` depends on strictly shorter intervals, so iterate by increasing length. Equivalently, iterate `l` from n−1 down to 0 and `r` from `l` up to n−1: then `dp[l][k]` (same row, smaller r) and `dp[k+1][r]` (larger l) are ready. Memoized recursion also works and avoids thinking about order, at some constant-factor cost.

For `a = 10 20 30 40` the table (rows l, columns r) is:

```
        r=0   r=1   r=2   r=3
l=0      0    30    90   190
l=1           0    50   140
l=2                0    70
l=3                     0
```

and the answer is `dp[0][3] = 190`.

The classical **matrix chain multiplication** problem has exactly the same structure, with cost `p[l]·p[k+1]·p[r+1]` for the last multiplication.

2.3 Games on a line: CSES 1097 *Removal Game*

Two players alternately remove the first or last number of a list and add it to their score; both play optimally. What is the first player's score?

Storing "the score of the player to move" requires knowing the totals of both players. A neater trick for two-player zero-sum games: let `dp[l][r]` be the **difference** (score of the player to move) − (score of the opponent) when only `x[l..r]` remains. If the mover takes `x[l]`, the opponent becomes the mover on `[l+1, r]` and achieves difference `dp[l+1][r]` in their favour, so the mover's difference is `x[l] − dp[l+1][r]`. Hence

```
dp[l][l] = x[l]
dp[l][r] = max(x[l] − dp[l+1][r], x[r] − dp[l][r−1])
```

Since first + second = total and first − second = `dp[0][n−1]`, the first player's score is `(total + dp[0][n−1]) / 2`. For `4 5 1 3`, `dp[0][3] = 3` and the answer is (13 + 3)/2 = 8. O(n²) for n ≤ 5000 — 25·10^6 `long long` cells is 200 MB, so on tight memory limits keep only two rows (the reference code uses the full table; CSES allows 512 MB).

AtCoder DP L *Deque* is the same problem asking for `X − Y` directly.

2.4 "Pair the first element with a later one": CF 1132F *Clear the String*

Delete a contiguous block of equal letters in one operation; minimize the number of operations to delete the whole string. Consider `s[l]`. Either it is deleted on its own (cost `1 + dp[l+1][r]`), or it is deleted in the same operation as some later `s[k] = s[l]`. In the latter case, everything strictly between `l` and `k` must disappear first (`dp[l+1][k−1]`), after which `s[l]` becomes adjacent to `s[k]` and is deleted together with it, so it costs nothing extra beyond `dp[k][r]`:

```
dp[l][r] = min(1 + dp[l+1][r],  min_{l < k ≤ r, s[k] = s[l]} dp[l+1][k−1] + dp[k][r])
```

with `dp` of an empty interval equal to 0. This "who is the first element grouped with?" pattern also solves Codeforces 607B *Zuma* and many palindrome-removal problems.

2.5 Pattern summary

Pattern	Transition	Examples
Last split	combine `dp[l][k]`, `dp[k+1][r]`	Slimes, matrix chain, optimal BST
Take an end	from `dp[l+1][r]`, `dp[l][r−1]`	Removal Game, Deque
Group first with k	`dp[l+1][k−1]` + `dp[k][r]`	Clear the String, Zuma
Matching ends	`s[l] == s[r]` → `dp[l+1][r−1]`	longest palindromic subsequence

Some O(n³) split DPs satisfy the quadrangle inequality and can be accelerated to O(n²) with Knuth's optimization (Nationals track, DP optimizations).

3. Digit DP

3.1 The setting

Count integers `x ∈ [A, B]` such that `P(x)` holds, where `B` may be 10^18 or a 10^4-digit string. We compute `f(N)` = number of `x ∈ [0, N]` with `P(x)` and answer `f(B) − f(A − 1)` (with `f(−1) = 0`; when A is a big string, compute `f(B) − f(A) + [P(A)]` instead of subtracting one).

3.2 The tight flag

Write `N` as digits `d₀ d₁ … d_{L−1}` (most significant first) and build `x` with exactly L digits, allowing leading zeros. At each position we choose a digit. As long as every digit chosen so far equals N's digit, we are **tight**: the next digit can be at most `d_pos`, and we stay tight only if we choose exactly `d_pos`. As soon as we choose a smaller digit, `x` is already less than `N` regardless of the remaining digits, and every later position is **free** (0–9).

The DP state is `(pos, state, tight)` plus, when needed, `started`:

- `state` is whatever the property needs to remember about the prefix: digit sum mod D, the last digit, the number of non-zero digits, the remainder mod M, a KMP automaton state, ….
- `started` tells whether a non-zero digit has been placed. Leading zeros are not real digits, so properties that look at specific digits ("no two adjacent digits equal", "count of zeros") must ignore them.

There is only one tight path (the digits of N itself), so memoizing only non-tight states loses nothing. A non-tight state's answer depends only on how many positions remain, not on N; but since the reference code indexes the memo by `pos` (whose meaning depends on N's length), it clears the memo at the start of every call to `f`. Indexing by "positions remaining" instead would let the memo be shared between `f(B)` and `f(A − 1)`.

3.3 Example: CSES 2220 *Counting Numbers*

Count integers in `[a, b]` (0 ≤ a ≤ b ≤ 10^18) with no two adjacent equal digits. State: last digit (10 = "none yet") and `started`. While not started, choosing 0 keeps us not started; choosing a non-zero digit starts the number. After starting, a digit equal to `last` is forbidden. When `pos == L` we have built one valid number (the all-zero string represents 0, which is valid). The sample `123 321` gives 171, and the reference code was checked against brute force on all ranges up to 3000.

Complexity: 19 positions × 11 last digits × 2 × 10 choices — a few thousand operations.

3.4 Example: AtCoder DP S *Digit Sum*

Count `x ∈ [1, K]` with digit sum divisible by D, where K has up to 10^4 digits and D ≤ 100, modulo 10^9+7. Leading zeros do not change the digit sum, so `started` is unnecessary. An iterative formulation is natural here: keep `dp[r]` = number of **free** prefixes of the current length with digit sum ≡ r (mod D), and the single tight prefix's digit sum `tightSum`. For each next digit `c` of K:

- every free prefix can be extended by any digit 0–9 and stays free;
- the tight prefix extended by a digit `d < c` becomes free;
- the tight prefix extended by `c` stays tight.

At the end, free prefixes with remainder 0 plus K itself (if its digit sum is divisible by D) are counted; subtract 1 for `x = 0`. Complexity O(|K|·D·10) = 10^7.

3.5 Variants

- **Remainder conditions**: carry `x mod M` (new remainder = `(10·r + d) mod M`). Codeforces 628D *Magic Numbers* combines this with digit-position constraints.
- **Bounded number of non-zero digits**: state = count so far (Codeforces 1036C *Classy Numbers*, AtCoder ABC 154 E *Almost Everywhere Zero*).
- **Substring constraints** ("does not contain 13"): state = KMP/automaton state (Week 28 prefix function, Week 29 Aho–Corasick).
- **Sum of all valid numbers** instead of count: carry (count, sum) pairs.

4. Reference implementations

All programs compile with `g++ -std=c++17 -O2 -Wall` and reproduce the official samples.

4.1 AtCoder DP N *Slimes*

```cpp
#include <bits/stdc++.h>
using namespace std;

// AtCoder DP Contest N - Slimes: merge adjacent slimes, cost = size of merged slime.
// dp[l][r] = min cost to merge a[l..r] into one slime. O(n^3).
int main() {
    int n;
    cin >> n;
    vector<long long> a(n), pre(n + 1, 0);
    for (int i = 0; i < n; i++) { cin >> a[i]; pre[i + 1] = pre[i] + a[i]; }
    const long long INF = LLONG_MAX / 4;
    vector<vector<long long>> dp(n, vector<long long>(n, 0));
    for (int len = 2; len <= n; len++) {              // increasing length!
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;
            dp[l][r] = INF;
            for (int k = l; k < r; k++)               // last merge: [l..k] + [k+1..r]
                dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r]);
            dp[l][r] += pre[r + 1] - pre[l];
        }
    }
    cout << dp[0][n - 1] << '\n';
}
```

4.2 CSES 1097 *Removal Game*

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 1097 - Removal Game: players alternately take the first or last number.
// dp[l][r] = (first player's score) - (second player's score) on x[l..r].
int main() {
    int n;
    cin >> n;
    vector<long long> x(n);
    long long total = 0;
    for (auto& v : x) { cin >> v; total += v; }
    vector<vector<long long>> dp(n, vector<long long>(n));
    for (int l = n - 1; l >= 0; l--) {                // l descending, r ascending
        dp[l][l] = x[l];
        for (int r = l + 1; r < n; r++)
            dp[l][r] = max(x[l] - dp[l + 1][r], x[r] - dp[l][r - 1]);
    }
    // first + second = total, first - second = dp  =>  first = (total + dp) / 2
    cout << (total + dp[0][n - 1]) / 2 << '\n';
}
```

4.3 CSES 2220 *Counting Numbers* (memoized digit DP)

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 2220 - Counting Numbers: how many integers in [a, b] have no two
// adjacent equal digits? Digit DP with memoization, answer = f(b) - f(a - 1).
string s;
long long memo[20][11][2];     // [pos][last digit or 10 = "none yet"][started]
bool seen[20][11][2];

// Count completions of positions pos.. given state; `tight` is not memoized.
long long go(int pos, int last, bool started, bool tight) {
    if (pos == (int)s.size()) return 1;               // a valid number (0 counts once)
    if (!tight && seen[pos][last][started]) return memo[pos][last][started];
    int limit = tight ? s[pos] - '0' : 9;
    long long res = 0;
    for (int d = 0; d <= limit; d++) {
        bool nt = tight && (d == limit);
        if (!started && d == 0) {
            res += go(pos + 1, 10, false, nt);        // still in leading zeros
        } else {
            if (started && d == last) continue;       // two adjacent equal digits
            res += go(pos + 1, d, true, nt);
        }
    }
    if (!tight) { seen[pos][last][started] = true; memo[pos][last][started] = res; }
    return res;
}
```

```
long long f(long long x) {                          // count in [0, x]
    if (x < 0) return 0;
    s = to_string(x);
    memset(seen, 0, sizeof seen);
    return go(0, 10, false, true);
}

int main() {
    long long a, b;
    cin >> a >> b;
    cout << f(b) - f(a - 1) << '\n';
}
```

4.4 AtCoder DP S *Digit Sum* (iterative digit DP)

```
#include <bits/stdc++.h>
using namespace std;

// AtCoder DP Contest S - Digit Sum: count x in [1, K] whose digit sum is
// divisible by D (K has up to 10^4 digits). Iterative digit DP, O(|K| * D * 10).
const int MOD = 1'000'000'007;

int main() {
    string K;
    int D;
    cin >> K >> D;
    // dp[r] = numbers of the current prefix length that are already strictly
    // below K's prefix ("free") with digit sum = r (mod D).
    vector<long long> dp(D, 0), ndp(D);
    int tightSum = 0;                                   // digit sum of K's prefix
    for (char ch : K) {
        int c = ch - '0';
        fill(ndp.begin(), ndp.end(), 0);
        for (int r = 0; r < D; r++) {                   // free -> free, any digit
            if (!dp[r]) continue;
            for (int d = 0; d <= 9; d++) {
                int nr = (r + d) % D;
                ndp[nr] = (ndp[nr] + dp[r]) % MOD;
            }
        }
        for (int d = 0; d < c; d++)                     // tight -> free, digit below c
            ndp[(tightSum + d) % D] = (ndp[(tightSum + d) % D] + 1) % MOD;
        tightSum = (tightSum + c) % D;
        swap(dp, ndp);
    }
    long long ans = dp[0] + (tightSum == 0 ? 1 : 0);  // free ones + K itself
    ans = (ans - 1 + MOD) % MOD;                        // remove x = 0
    cout << ans << '\n';
}
```

5. Common mistakes

Interval DP

- **Wrong fill order.** Plain row-major loops (`l` ascending, `r` ascending) read `dp[l+1][…]` before it exists. Iterate by length, or `l` descending.
- **Memory.** `long long dp[5000][5000]` is 200 MB. Use `int` when possible or keep two rows when the transition only touches `l` and `l+1`.
- **Off-by-one in sums.** With `pre[i] = a[0] + … + a[i−1]`, `sum(l, r) = pre[r+1] − pre[l]`.
- **Applying greedy** where only DP is correct (the Slimes counterexample above).

Digit DP

- **Memoizing tight states** and then reusing the memo for a different bound. Memoize only non-tight states, or clear the memo per call.
- **Leading zeros** treated as digits ("007" has adjacent equal digits only if zeros count). Carry `started` when the property inspects individual digits.
- **Range endpoints.** `f(B) − f(A − 1)`, with `A = 0` handled (`f(−1) = 0`). If the problem counts positive integers, remove `x = 0`.
- **Big inputs as strings.** When B has thousands of digits, read it as a string and avoid converting to integers.
- **Modulo** in counting variants: reduce after every addition.

6. Practice problems

Warm-up:

- AtCoder DP Contest L — *Deque* — https://atcoder.jp/contests/dp/tasks/dp_l
- CSES 1097 — *Removal Game* — https://cses.fi/problemset/task/1097
- AtCoder DP Contest N — *Slimes* — https://atcoder.jp/contests/dp/tasks/dp_n

Core:

- CSES 1744 — *Rectangle Cutting* — https://cses.fi/problemset/task/1744
- Codeforces 1132F — *Clear the String* — https://codeforces.com/problemset/problem/1132/F
- AtCoder DP Contest S — *Digit Sum* — https://atcoder.jp/contests/dp/tasks/dp_s
- CSES 2220 — *Counting Numbers* — https://cses.fi/problemset/task/2220
- Codeforces 1036C — *Classy Numbers* — https://codeforces.com/problemset/problem/1036/C

Challenge:

- Codeforces 607B — *Zuma* — https://codeforces.com/problemset/problem/607/B
- Codeforces 1114D — *Flood Fill* — https://codeforces.com/problemset/problem/1114/D

- Codeforces 628D — *Magic Numbers* — https://codeforces.com/problemset/problem/628/D
- AtCoder ABC 154 E — *Almost Everywhere Zero* — https://atcoder.jp/contests/abc154/tasks/abc154_e

7. Further reading

- CP-Algorithms, "Knuth's Optimization" (for later) — https://cp-algorithms.com/dynamic_programming/knuth-optimization.html
- USACO Guide (https://usaco.guide), modules "Range DP" and "Digit DP".
- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 7 "Dynamic programming".
- AtCoder Educational DP Contest editorial discussions (tasks L, N, S) — https://atcoder.jp/contests/dp
- Next week: Strings I (hashing, prefix function, Z-function). The prefix-function automaton combines with digit DP for "number does not contain pattern" problems.